

UNIVERSIDAD DE MONTERREY
Escuela de Ingeniería y Tecnología
Ingeniería en Tecnologías Computacionales

Decisiones Ingenieriles

Autor(es)
600046 Fernando Olivares Del Valle

Profesor (a)
Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

Decisiones Ingenieriles

Decisión 1: Arquitectura monolítica server-side

Necesidad/problema: Se requiere construir un sistema de gestión de librería que sea sencillo de desarrollar, desplegar y mantener por un solo estudiante en el marco de un ejercicio académico, sin la complejidad operativa de administrar múltiples servicios.

Alternativas consideradas:

- Arquitectura de microservicios (un servicio por dominio: usuarios, libros, imágenes).
- Backend desacoplado con API REST/GraphQL + frontend SPA (React/Vue).
- Monolito server-side con renderizado HTML en el servidor (Node.js + Express + EJS).

Decisión tomada: Monolito server-side con Node.js, Express y EJS, según lo requerido explícitamente por el ejercicio.

Justificación técnica: Un solo proceso desplegable reduce la complejidad de infraestructura (no hay que orquestar múltiples servicios, balanceadores ni comunicación entre ellos), simplifica el despliegue con un único reverse proxy, y es apropiado para el tamaño del equipo (un desarrollador) y el volumen de datos (uso académico/demo).

Riesgo o limitación: Escalar el sistema a mayor tráfico o equipos de desarrollo grandes se vuelve más difícil, ya que todos los módulos comparten el mismo proceso y despliegue; un error en un módulo puede afectar la disponibilidad de toda la aplicación.

Evidencia de validación: La aplicación corre como un único proceso Node.js (app.js) escuchando en 127.0.0.1:3000, publicado mediante un solo reverse proxy (Apache o NGINX), sin servicios adicionales desplegados de forma independiente.

Decisión 2: Acceso directo a PostgreSQL con consultas parametrizadas

Necesidad/problema: El sistema necesita persistir y consultar datos relacionales (libros, autores, géneros, conceptos, usuarios) con integridad referencial y protección contra inyección SQL.

Alternativas consideradas:

- ORM (Sequelize, Prisma) para abstraer las consultas.
- Acceso directo a PostgreSQL mediante el driver **pg** con consultas SQL parametrizadas.
- Query builder intermedio (Knex).

Decisión tomada: Acceso directo a PostgreSQL con el paquete **pg**, usando exclusivamente consultas parametrizadas (placeholders \$1, \$2), sin ORM.

Justificación técnica: El ejercicio exige control explícito y comprensión directa del SQL ejecutado (requisito de la restricción arquitectónica), evita la sobrecarga de aprendizaje y configuración de un ORM, y las consultas parametrizadas de **pg** ya protegen contra SQL Injection sin necesitar una capa de abstracción adicional.

Riesgo o limitación: Sin un ORM, hay más código repetitivo en **models** y mayor responsabilidad manual de mantener las consultas sincronizadas con el esquema de la base de datos; un cambio en el esquema requiere actualizar manualmente cada consulta afectada.

Evidencia de validación: Todas las consultas en `src/models/` usan parámetros (`$1`, `$2`, ...) en lugar de concatenación de strings; una prueba con caracteres especiales (comillas, `--`, `;`) en un campo de entrada no altera la consulta ejecutada.

Decisión 3: Renderizado server-side con EJS

Necesidad/problema: El sistema necesita presentar información al usuario (catálogo, formularios, detalle de libros) sin usar APIs REST/GraphQL ni JSON/XML como intercambio de datos, según restricción explícita del ejercicio.

Alternativas consideradas:

- Frontend separado (SPA) consumiendo una API JSON.
- Renderizado server-side con un motor de plantillas (EJS, Pug, Handlebars).

Decisión tomada: Renderizado server-side con EJS, embebido directamente en Express.

Justificación técnica: EJS permite generar HTML dinámico usando JavaScript nativo dentro de las plantillas, se integra de forma directa con Express sin dependencias adicionales, y cumple con la restricción de no usar JSON/XML como mecanismo de intercambio frontend-backend, ya que los formularios HTML envían los datos directamente al monolito vía **POST**.

Riesgo o limitación: La interactividad del lado del cliente es limitada comparada con un frontend SPA (cada acción implica una recarga completa de página), y mezclar lógica de presentación con JavaScript embebido en las vistas puede tentar a incluir lógica de negocio si no se disciplina la separación de responsabilidades.

Evidencia de validación: Las vistas en `src/views/` reciben los datos ya procesados desde **controllers/** y solo los recorren para generar HTML, sin contener consultas SQL ni llamadas directas a **models/**.